

Phase 2 Real-World Flight Data Research Findings

What Phase 2 needs to succeed

Phase 2 stops being “a pretty graph” once the underlying data supports real decisions. That requires three things.

First, timely operational signals. You need scheduled times, estimated times, actual gate and runway timestamps where available, plus cancellation and diversion status. This is the minimum needed to compute delay buildup and spillover across a network. ¹

Second, predictable access rules. An app that “helps for real” needs stable quotas, known throttles, and license terms that match the intended use. OpenSky’s official documentation frames its live API as research and non-commercial, and explicitly notes it does not provide schedules or delays beyond what ADS-B data implies. ²

Third, a way to create your own historical record. Several providers offer limited historical windows by default, so Phase 2 needs to ingest continuously and store in your own database to build a growing dataset over time. Aviationstack states its historical coverage as a rolling three-month window, which makes self-storage essential once you want longer trendlines. ³

Flight data source evaluation

Aviationstack is a straightforward REST option for flight status plus schedules, with published plan tiers and an explicit commercial license on paid plans. The pricing page lists a free plan (100 requests/month) and paid tiers starting at \$49.99/month for 10,000 requests. It also states real-time status updates lag by roughly 30–60 seconds, and historical flight data is available for the last 3 months as a sliding window. ³

The documentation shows separate throttles for schedule-like endpoints such as `/timetable` and `/flightsfuture`, with free plans limited to 1 request per 60 seconds and paid plans limited to 1 request per 10 seconds for those endpoints. That matters if your product relies on polling many airports frequently. ⁴

OpenSky Network is best understood as ADS-B state vectors and track geometry, not a commercial schedules-and-delays feed. OpenSky’s own docs emphasize state vectors as the core concept and explicitly note the API does not provide commercial flight data like airport schedules or delays. ⁵

Its REST API documentation lays out strict daily “API credits” and time-resolution rules. Anonymous access is limited to the most recent state vectors, 10-second time resolution, and 400 API credits/day. Authenticated users get 5-second resolution, a limited lookback for some calls, and 4,000 API credits/day, with up to 8,000/day for active contributing users. ⁶

This is excellent for live fleet tracking features, and for generating real coordinate trails, but it does not solve “delay propagation” on its own. ²

FlightAware's AeroAPI is positioned as a paid commercial product with per-query usage fees. The AeroAPI page shows the "Standard" tier has a \$100/month minimum and "Premium" has a \$1,000/month minimum, with a "Personal" tier that has no monthly minimum but tight throughput limits (10 result sets per minute). The same page also lists which features are included in each tier, including historical flight data only on Standard and Premium. ⁷

This makes it credible for production systems, but it pushes Phase 2 toward a cost-floor that is hard to justify without a clear revenue path. ⁷

Flightradar24 has an official API offering and publicly documents that you must subscribe to access production endpoints, with a sandbox option that returns static responses for schema testing. ⁸

Their public blog post about getting started with the API references an entry "Explorer" option at \$9/month and explains the product in terms of credit-based usage, including a time-limited promotion structure (which signals that credit amounts and promos change over time). ⁹

Their Terms of Service also describe three API subscription levels: Explorer, Essential, and Advanced. ¹⁰

That said, the key decision point is not brand recognition. It is whether the offered endpoints cleanly support the Phase 2 use case without forcing high-frequency polling. The public sources above confirm a credit-based model and subscription gating, but they do not disclose enough public detail about the specific delay-centric endpoints to treat it as the safest first dependency for Phase 2. ¹¹

Cirium's developer center presents a credible "enterprise-grade" posture and offers an evaluation plan for 30 days, with a published evaluation usage cap of 20,000 total transactions (lifetime) and additional per-API caps for certain features. It also states that commercial access uses volume-based pricing. ¹²

Cirium also documents the Flight Status Feed API as a stream of worldwide flight status records updated frequently, with dataset refreshes about every six seconds on average, and a throttle guardrail of 3,000 requests per minute per user account. ¹³

The limitation: detailed pricing is behind an authenticated page, which blocks a full cost/rate analysis without an account. ¹⁴

Amadeus Self-Service is strong for travel-data workflows, but its own FAQ documents a major dataset gap: it does not return data on American Airlines, Delta, British Airways, and low-cost carriers in the Self-Service dataset. That missing coverage breaks the usefulness of a U.S. network reliability product, since those airlines represent a large fraction of U.S. traffic. ¹⁵

Amadeus also publishes rate limits per environment, including up to 40 transactions per second per user in production for most Self-Service APIs. ¹⁶

Even with good throughput, the carrier coverage gap is the blocker for this Phase 2 project direction. ¹⁵

Recommended API for Phase 2

Phase 2 needs a dependable "status + schedule" backbone with clear pricing and a license that supports building a public-facing application.

Recommendation: use Aviationstack as the Phase 2 primary API.

This recommendation rests on three concrete advantages.

Aviationstack publishes pricing tiers with explicit request volumes and distinguishes personal vs commercial licensing in its plans, which reduces procurement uncertainty. ³

It explicitly markets real-time flights, historical flights, and schedules as first-class features, which matches the minimum dataset needed for delay propagation analytics and route-network computation. ¹⁷

It openly states a rolling historical window of three months and near-real-time delay of 30–60 seconds, which is a manageable constraint for a Phase 2 product once you store ingested results into your own database. ³

What Aviationstack is not: a high-frequency, per-airport timetable scraper. The timetable and future-schedule endpoints have separate throttles, including 1 request per 10 seconds on paid plans. That rate is workable, but only if Phase 2 designs around caching and batching rather than naïve polling of every hub. ⁴

A practical Phase 2 rule: use Aviationstack for event updates and per-flight details, not for constantly rebuilding the entire country’s schedule graph from scratch every minute. ¹⁸

OpenSky fits as an optional enrichment source for map tracks in a “research/demo mode,” not as the production backbone. OpenSky’s own documentation states non-commercial intent and explicitly excludes schedules and delay products. ²

Primary real-world use case

Primary use case: Delay propagation analysis for the next 24 hours.

This is the best Phase 2 anchor because it aligns with what you already built (a network visualization) while staying realistic about what flight APIs actually deliver.

Delay propagation is a real, studied phenomenon. Research on the U.S. airport network describes how initial delays propagate and spread through the network as operations continue. ¹⁹

More recent work continues to emphasize spatiotemporal cascading effects and frames propagation forecasting as an operational resilience problem, especially during extreme weather. ²⁰

This use case also maps cleanly to available data.

U.S. Bureau of Transportation Statistics ²¹ data under 14 CFR Part 234 provides a long-running historical baseline, with data elements such as scheduled vs actual times and delay measures, and availability cited as far back as 1987 for some series. That dataset is ideal for validating your propagation metrics and calibrating expected cascade patterns even if Phase 2 ships with “near-real-time” rather than “instant” updates. ²²

A 2024 audit report from the U.S. Department of Transportation Office of Inspector General ²³ discusses BTS controls around verifying accuracy and highlights opportunities to improve completeness and consistency checks. That is a reminder to treat any single data stream as imperfect and design Phase 2 with sanity checks and anomaly detection. ²⁴

Passenger route recovery is valuable, but it breaks down fast without inventory, fare rules, and seat availability. Airline rebooking decisions depend on constraints that most “flight status” APIs do not include. That makes it a better Phase 3 expansion once Phase 2 proves real operational insight. ²⁵

Environmental impact and carbon routing is also valuable, but it requires aircraft type, distance, occupancy assumptions, and emissions factors. That introduces modeling debates and user trust risk before Phase 2 has a stable operational core. ²⁶

Live fleet tracking is fun and visually compelling, but it tends to help enthusiasts more than decision-makers. It also pulls the product toward heavy real-time path rendering, which is the exact complexity-trap you flagged. ²⁷

Flight path vs node connection data

Phase 2 should draw straight node connections, not full coordinate trails.

A delay propagation product needs topology clarity, not cinematic tracks. Straight connections communicate “where delays flow” with far less data and render load.

Real coordinate trails require high-frequency geospatial sampling and a track endpoint with rate constraints and daily credit budgets. OpenSky’s REST docs include a dedicated “Track by Aircraft” endpoint and define API-credit constraints and time-resolution rules that directly impact how detailed a trail you can pull across many aircraft in parallel. ⁶

If you try to run network-wide tracking at scale, you either hit aggressive quota limits or move into enterprise pricing tiers with providers like Cirium or FlightAware. Cirium also documents throttling mechanisms even on feed-style products, and FlightAware quickly steps into monthly minimums for higher-throughput tiers. ²⁸

The better Phase 2 compromise: keep straight lines for the network, and add “selective paths” only for deep dives.

Example: when a user clicks a disrupted hub, show straight-line edges for network impact, then show 1–3 representative live tracks as an optional overlay for explanation, not as the default rendering mode. This pattern matches OpenSky’s strengths without making your whole UI depend on continuous track ingestion. ²⁹

Data storage and ingestion architecture

Your “Supabase or something more efficient” instinct is right. Phase 2 needs a real database, not browser memory.

Use Supabase ³⁰ as the Phase 2 data backbone, because it provides Postgres, authentication, row-level security, and real-time change delivery in one stack. ³¹

A workable Phase 2 pipeline looks like this.

Ingest external flight updates into a Postgres table that records immutable events, then derive current state with a view or a materialized view. This design avoids overwriting facts and helps debug “why did this flight look delayed at 2:10 PM.” Aviationstack’s three-month rolling history limit strengthens the case for storing every update you ingest, so you own the long-term record. ³

Schedule ingestion jobs inside Postgres using the `pg_cron` extension, which Supabase documents as a way to schedule recurring jobs with cron syntax. ³²
Then trigger outbound HTTP pulls or function calls via server-side compute. Supabase documents Edge Functions as server-side TypeScript functions built on Deno, which fits a clean ingestion-worker pattern. ³³

Lock down read/write paths with Postgres Row Level Security. Supabase docs emphasize enabling RLS for any tables in exposed schemas and framing policies as row-level filters. That matters once your app has user accounts, saved dashboards, alerts, and shared workspaces. ³⁴
For UI freshness, Supabase documents “Postgres Changes” subscriptions to listen to database events. That supports near-real-time dashboard updates without reopening HTTP loops from the browser. ³⁵

This architecture also keeps you flexible on data providers. If the Phase 2 API choice changes later, your product stays stable because your database schema and derived metrics stay stable.

Phase 2 name choice

New name: HubStream.

It matches the product direction. It signals flow through hubs and network effects. It stays aligned with delay propagation as the primary value. ³⁶

¹ ³ ¹⁷ ¹⁸ ²⁶ <https://aviationstack.com/pricing/>
<https://aviationstack.com/pricing/>

² ⁵ ²⁵ <https://openskynetwork.github.io/opensky-api/>
<https://openskynetwork.github.io/opensky-api/>

⁴ <https://aviationstack.com/documentation/>
<https://aviationstack.com/documentation/>

⁶ ²⁷ ²⁹ <https://openskynetwork.github.io/opensky-api/rest.html>
<https://openskynetwork.github.io/opensky-api/rest.html>

⁷ <https://www.flightaware.com/commercial/aeroapi>
<https://www.flightaware.com/commercial/aeroapi>

⁸ <https://support.fr24.com/support/solutions/articles/3000128177-can-i-test-the-api-before-purchasing-a-subscription->
<https://support.fr24.com/support/solutions/articles/3000128177-can-i-test-the-api-before-purchasing-a-subscription->

⁹ ¹¹ <https://www.flightradar24.com/blog/b2b/how-to-get-started-with-the-flightradar24-api/>
<https://www.flightradar24.com/blog/b2b/how-to-get-started-with-the-flightradar24-api/>

10 <https://mobile.flightradar24.com/mobile/terms>

<https://mobile.flightradar24.com/mobile/terms>

12 <https://developer.flightstats.com/getting-started/>

<https://developer.flightstats.com/getting-started/>

13 28 <https://developer.flightstats.com/api-docs/flight-status-feed/v2/flight-status-feed-api>

<https://developer.flightstats.com/api-docs/flight-status-feed/v2/flight-status-feed-api>

14 <https://developer.flightstats.com/getting-started/pricing>

<https://developer.flightstats.com/getting-started/pricing>

15 <https://developers.amadeus.com/support/faq/about-self-service-apis>

<https://developers.amadeus.com/support/faq/about-self-service-apis>

16 <https://developers.amadeus.com/self-service/apis-docs/guides/rate-limits-745>

<https://developers.amadeus.com/self-service/apis-docs/guides/rate-limits-745>

19 36 [Systemic delay propagation in the US airport network | Scientific Reports](https://www.nature.com/articles/srep01159?utm_source=chatgpt.com)

https://www.nature.com/articles/srep01159?utm_source=chatgpt.com

20 30 [A Data-Driven Framework for Flight Delay Propagation Forecasting During Extreme Weather](https://www.mdpi.com/2227-7390/13/21/3551?utm_source=chatgpt.com)

https://www.mdpi.com/2227-7390/13/21/3551?utm_source=chatgpt.com

21 32 <https://supabase.com/docs/guides/database/extensions/pgcron>

<https://supabase.com/docs/guides/database/extensions/pgcron>

22 [Airline Service Quality Performance 234 \(On-Time performance data\) | Bureau of Transportation Statistics](https://www.bts.gov/browse-statistical-products-and-data/bts-publications/airline-service-quality-performance-234-time?utm_source=chatgpt.com)

https://www.bts.gov/browse-statistical-products-and-data/bts-publications/airline-service-quality-performance-234-time?utm_source=chatgpt.com

23 33 <https://supabase.com/docs/guides/functions>

<https://supabase.com/docs/guides/functions>

24 [The Bureau of Transportation Statistics Verifies the Accuracy of Flight Delay and Cancellation Data but Can Do More To Assess Its Completeness and Consistency | DOT OIG](https://www.oig.dot.gov/library-item/46490?utm_source=chatgpt.com)

https://www.oig.dot.gov/library-item/46490?utm_source=chatgpt.com

31 34 <https://supabase.com/docs/guides/database/postgres/row-level-security>

<https://supabase.com/docs/guides/database/postgres/row-level-security>

35 <https://supabase.com/docs/guides/realtime/postgres-changes>

<https://supabase.com/docs/guides/realtime/postgres-changes>